ModelAnvil Design Brief

Version 0.1 | 8 October 2026 | PRD baseline 0.7 | Status proposed visual and interaction direction

Design ModelAnvil as a focused technical workspace where teams can inspect data, review model suggestions, run experiments, and make evidence-based release decisions. The interface must remain approachable for annotators and data engineers while giving researchers access to exact configuration and lineage. The primary design outcome is a coherent workflow across these roles.

This brief specifies product design direction and handoff requirements. It is not a completed screen design or a usability-validated brand system. The initial ICP remains AI product companies with dedicated ML teams; enterprise platform and research teams are secondary audiences to validate through pilots.

1 Audience and core jobs

Role	Job to complete	Design emphasis
Researcher or ML engineer	Train, compare, diagnose, and release a model	Reproducible inputs, metric context, efficient advanced controls
Data engineer	Acquire, clean, map, split, and publish data	Source preview, affected examples, provenance, failures
Annotator or reviewer	Produce accurate approved labels	Fast keyboard workflow, visible saves, clear rubric, fair comparison
Application engineer	Build RAG and test context	Exact supplied input, retrieved evidence, traces, controlled experiments
Evaluation lead	Identify regressions and decide readiness	Slices, sample counts, uncertainty, gates, independent evidence
Administrator or operator	Connect resources and recover work	Ownership, capacity, policy, actual execution state

2 Product principles

Keep organization/project identity visible throughout work. Show the selected immutable version near any dataset, model, recipe, or evaluation. Link source → transformation → label → snapshot → run → result → release through inspectable lineage.

Use progressive disclosure: common recipes first, an advanced editor for full configuration, and an exportable configuration for automation. Both views operate the same record. Users should see a concrete reason for an unavailable action and an authorized way to resolve it.

Preserve uncertainty. Separate unknown estimates from measured usage, missing metrics from zero, model confidence from verified quality, model-origin acceptance from independent human authorship, and projected visual distance from actual embedding similarity.

Show state transitions honestly. Cancellation requested, backend stopped, and resources released are distinct. An imported file can be stored while preview or training remains unsupported. A model can finish training while failing release criteria.

3 Visual direction

Use a quiet, precise workspace with neutral surfaces, strong text hierarchy, and restrained blue for navigation and primary actions. Avoid decorative industrial textures or anvil imagery in dense product screens. The ModelAnvil name can inform the wordmark; the symbol and brand identity remain a separate design exercise.

Token group	Initial design proposal
Background	Canvas #F6F8FB; surface #FFFFFF; raised surface with subtle border
Text	Primary #172033; secondary #475569; muted only after contrast validation
Borders	#CBD5E1 for controls; #E2E8F0 for nonessential dividers
Primary action	Blue #1D4ED8; hover #1E40AF; white text
Status	Success #166534; caution #92400E; error #B91C1C; information blue
Type	System UI sans; native SF on Mac; system monospace for code, hashes, and exact inputs
Scale	Body 14-16 px, labels 12-14 px, section 18-20 px, page title 24-28 px; code 13-14 px
Spacing	4 px base with 8, 12, 16, 24, 32 px steps
Shape	6-8 px control/card radius; minimal shadows; visible focus ring

These colors are starting tokens, not a contrast certification. Validate each actual foreground/background/state combination. Supply light and dark themes through semantic tokens; do not invert plots or source media blindly. Persist user preference. Status always includes text/icon, not color alone.

4 Layout and density

Desktop web uses a proposed 224-256 px navigation rail, contextual page header, main content, and optional 320-420 px inspector. At narrower widths, the inspector becomes a drawer and navigation collapses. Primary builders are designed for desktop; smaller screens retain monitoring, approval inspection, and readable tables without squeezing every builder panel onto one screen. Breakpoints require prototype validation, with 1440, 1024, and 768 px reference layouts.

Use a list/detail pattern for registries, table plus inspector for jobs and evaluations, and split source/editor for annotation. Full-width charts or exact input previews can temporarily expand. Preserve the selected resource and version when opening/closing inspectors. Do not represent every row as a separate card.

Tables support sticky headers, explicit units, configurable columns, bounded virtualization, keyboard selection, sorting, and saved filters. Long hashes shorten visually with accessible copy/full detail. Do not truncate the only explanation of a failure. Large payloads load on demand within authorized scope.

5 Required component system

Component	Required behavior
Organization/project selector	Clear active context; access-safe switching; remembers permitted selection
Version chip and lineage panel	Stable version, readiness, ancestry, exact input diff, copy/reference
Capability badge	Separate storage/ parse/ preview/ annotation/ train/ export states with reason
Job status and timeline	Stage, actual state, progress, unknown ETA, retry/cancel/resume controls
Preflight report	Blocking errors, warnings, measured/estimated resources, pilot, review launch
Source and sample viewer	Original/extracted split, positions, media controls, metadata, failures
Annotation editor	Task-specific controls, rubric, shortcuts, save state, diff, conflict, escalation
Blind ranking panel	Randomized candidate order, ties/abstain, no identity leakage during review
Campaign timeline	Batch/model/snapshot linkage, independent quality, corrections, limits
Metric card/table	Definition, cutoff/unit, sample count, slices, uncertainty, N/A and failure count
Trace inspector	Retrieval → reranking → supplied context → answer/citations with timings
Context block editor	Typed blocks, provenance/trust, budget bars, exact serialization and omission diff
Cluster explorer	Search/filter/lasso, sampling metadata, query overlay, original neighbors, table equivalent
Release gate panel	Frozen evidence/rules, pass/fail/missing, authorized exception, rollback target

Components share terminology and states across modules. Web implementation should establish an accessible component foundation during WP-02; library selection is an engineering/design decision after prototype and license review. Do not assume a chosen component library guarantees accessibility.

6 Key screen composition

Overview: goals and next prerequisites first, then active work, recent qualified results, and usage. Avoid presenting raw count of training runs as project success.

Data preparation: source sample on the left, recipe steps in the center, affected counts/reasons on the right. Preview sample scope is always visible. Publishing shows schema, split integrity, licenses, and immutable version.

Annotation: content and rubric are primary; draft labels and corrections are inspectable without consuming all reading space. Sticky action bar includes save, accept/correct, skip, and escalate. Independent audit mode hides draft predictions. Reviewers see conflicts without losing their draft.

Training: guided recipe configuration, exact-version summary, compatibility/preflight, and cost assumptions precede launch. Monitoring prioritizes stage and health, then comparable loss/validation, throughput, checkpoints, and usage. Log detail is secondary and permission-bound.

RAG workbench: query and answer sit beside a step trace. Retrieval scores, relevance labels, actual supplied chunks, and citations have distinct styling and semantics. The explorer shares selection with a table/failure panel; a visually isolated cluster is not labeled a quality improvement.

Context Lab: block list/composer, exact messages, token allocation, and experiment configuration. Results offer paired case inspection and quality/token/cost charts. Display removed/transformed blocks and memory/tool fixture identity.

Release review: candidate, baseline, required evidence, failed/missing checks, and compatible rollback. The primary action reflects actual readiness and the viewer's authority.

7 Accessibility and interaction quality

Use WCAG 2.2 AA as the proposed web acceptance target and native accessibility conventions on Mac. Validate keyboard operation, semantic names/roles, logical focus order, visible focus, screen-reader updates, reflow, contrast, zoom, reduced motion, and non-color status cues. Complex plots have equivalent tables and text summaries. Media annotation supports applicable transcript/controls and task-specific accessible alternatives.

Prefer 44 px interaction targets where practical; compact expert tables must retain a tested accessible target policy. Shortcuts are documented, configurable where they conflict, and disabled while typing when appropriate. Timed assignments warn before expiry and preserve unsent drafts. Autosave/error announcements should be useful without flooding assistive output.

Annotation shortcuts must never turn an accidental keypress into an irreversible batch approval. Destructive changes, chargeable launches, and publishing use concrete scope review. Model text and scraped markup render as untrusted content in isolated viewers; interface content cannot execute imported scripts.

8 Microcopy and feedback

Use plain states such as Ready for review, Needs adjudication, Candidate failed quality gate, Waiting for compatible GPUs, Cancellation requested, and Stopped. Explain the affected object and next action: “Three classes have no approved examples. Add labels before training.” Do not show a generic “Something went wrong” as the only recovery information.

Differentiate model draft, human correction, approved model-assisted label, and independent audit label. Say “Estimated cost” with assumptions, “Unavailable” for missing measurements, and “N/A - no labeled relevant evidence” where appropriate. Show reproducible random sampling settings and declared projection coverage in inspectable detail.

9 Mac experience

Use native navigation, file dialogs, keyboard menus, accessibility APIs, secure credential storage, and background transfer/reconnect behavior. Share semantic design tokens and terminology with web while following native conventions. The Mac app is an API client; it does not imply local GPU training. Proposed feature coverage is recorded in the flow document and must be settled before commitment. Unsupported advanced views can open an authenticated browser route with the same resource/version.

10 Design deliverables and acceptance

Produce an information architecture map, state inventory, annotated low-fidelity flows, component/token library, responsive high-fidelity screens, interactive prototypes, and engineering handoff annotations. Prioritize both labeling seeds, scratch and fine-tuning launch, RLHF evidence, RAG trace/explorer, context comparison, and failing release gates.

Run formative sessions with researcher, data engineer, annotator, application engineer, and operator representatives from pilot organizations. Test completion and misunderstanding, not just visual preference. Record task success, blocker frequency, time/effort, recovery behavior, and terminology issues; establish baselines before setting numeric productivity goals.

Design review passes when critical journeys cover empty/loading/partial/error/permission/conflict/offline states, keyboard-only annotation works, version/provenance is visible, comparisons cannot hide missing evidence, and prototypes hand off explicit permissions and lifecycle behavior. Brand assets, exact layout widths, and numeric usability targets remain proposals until reviewed.